

Homework 8 - Discussion

For this Ben Olschar, 108021211678 and Frederik Hüttemann, 108021215247 cooperated with each other. We did implement the code on our own, but agreed to using one version. The discussion and results are made in cooperation.

Task Configuration

For the GPU algorithms, we needed the `BLOCK_SIZE` and `GRID_SIZE` to launch the kernels. The `BLOCK` and `GRID_SIZE` for the cascaded algorithm from HW7 could be copy-pasted as they were already calculated in the last exercise.

To launch the kernel for the HW8 kernel, we need to calculate the number of active block on our GPU. For this the GPU config has been read using `cudaGetDeviceProperties`. First of all, we fixed the number of persistent blocks approximately equal to the maximum number of concurrently active block. Therefore `BLOCK_SIZE = 256`;

The `GRID_SIZE` is the calculated using the following formula, to maximize the number of activate threads:

$$\text{GRID_SIZE} = \text{number of SMs} \cdot \frac{\text{max threads per SM}}{\text{BLOCK_SIZE}} = 34 \cdot \frac{1536}{256} = 204$$

Therefore, we set `GRID_SIZE = 204`.

Task 1: CPU reduction algorithm

Here, a baseline on CPU is implemented used to compute the results. The reduction algorithm simply returns the sum of the array. The CPU is the slowest test because of sequential operations. The code has been copied from the last HW7 to maintain correctness.

Task 2: GPU reduction using cascaded algorithm (HW7)

I called this task 2, as this is also part of the code base. Here, we copy-pasted the algorithm from last homework to create a competitive result the Harris algorithm can compete against. As already stated in the last homework, the algorithm is significantly faster the the sequential CPU algorithm as this makes use of parallelism and tries to avoid blocking usage of `atomicAdd`.

Task 3: Harris Cascaded Algorithm with threadfence (HW8)

This is the main task of the homework. Here, the optimizations should be implemented, presented in the lecture. Because some of the optimizations of the lecture were even more optimized by later suggestions, we only took the following optimizations: - **Reduction #3: Sequential Addressing**: this optimization tries to bank conflicts on the shared memory (this is also an optimized reduction for #1 and #2, as these reductions #1 and #2 suffer under bank conflicts). - **Reduction #5: Unroll last warp**: improves speed by removing unnecessary thread synchronization within a warp - **Reduction #7: Multiple Adds per Thread**: improves loading the data by already performing the first addition. (this optimizes the Reduction #4 where only a single addition is performed on loading) - **__threadfence**: improves addition on the final reduction

What we expected

Because the last week's algorithm is already pretty optimized, no real speedup was expected. the Harris algorithm should be a bit faster, but its speed is still limited by hardware reads/writes.

The atomic cascaded algorithm took 0.32 ms on average to reduce the 40 MB float. The implementation of this week's homework is roughly 1.4 times faster. This is because of the reductions of sequential addressing, getting rid of unnecessary threadsyncs and using threadfence in the end. Overall, the algorithm is more optimized and therefore the speedup is expected.

Because the HW7 algorithm is already pretty good, the speedup of x1.4 is great, as the number of additions for both algorithms is the same. The speedup only comes from optimizing memory access and removing unnecessary kernel functions (threadsync).